RUNGTA INTERNATIONAL SKILLS UNIVERSITY
School of Computer Science and Engineering
B.Tech Computer Science and Engineering - First Semester

Python Project Report: Employee Leave Management System

Problem Statement No.	60	**Language**	Python 3
Project Title	Employee Leave Management System		
Submitted By	Amit Kumar Gupta	**Concepts Used**	Dictionaries, Functions, Conditions, Dates

1. Problem Statement

Develop a console application in Python 3 to manage employee leave. An employee applies for leave of a given type between two dates. The program calculates the number of days (inclusive of both dates), checks if enough leave balance is available, and approves or rejects the request. Employee E101 (Amit) has a yearly balance of 12 Casual, 10 Sick, and 15 Earned leaves.

2. Logic Used

- Employee data and leave balances are stored in a nested dictionary named `employees`.
- The `datetime` module is used to convert input dates (`YYYY-MM-DD`) and find total days using `(d2 - d1).days + 1`.
- Two functions are created: `apply_leave()` to check conditions and update leave balance, and `view_balance()` to print remaining leaves.
- A `while True` loop runs the console menu until the user selects option 3 to exit.

3. Source Code (leave_management.py)

```
from datetime import datetime

employees = {
    "E101": {
        "name": "Amit",
        "Casual": 12,
        "Sick": 10,
        "Earned": 15
    }
}

def apply_leave():
    emp_id = input("Enter Employee ID: ")
    if emp_id not in employees:
        print("Invalid Employee ID")
        return

    l_type = input("Enter Leave Type (Casual/Sick/Earned): ")
    if l_type not in ["Casual", "Sick", "Earned"]:
        print("Invalid Leave Type")
        return

    f_date = input("Enter From Date (YYYY-MM-DD): ")
    t_date = input("Enter To Date (YYYY-MM-DD): ")

    try:
        d1 = datetime.strptime(f_date, "%Y-%m-%d")
        d2 = datetime.strptime(t_date, "%Y-%m-%d")
    except:
        print("Invalid date format")
        return

    if d2 < d1:
        print("Invalid dates. To-date cannot be before from-date.")
        return

    days = (d2 - d1).days + 1
    bal = employees[emp_id][l_type]

    if days <= bal:
        employees[emp_id][l_type] = bal - days
        print("Leave approved:", days, "days", l_type, "| Balance", l_type + ":", e
    else:
        print("Leave rejected. Requested:", days, "days | Available:", bal, "days")

def view_balance():
    emp_id = input("Enter Employee ID: ")
    if emp_id not in employees:
        print("Invalid Employee ID")
        return

    c = employees[emp_id]["Casual"]
```

4. Test Cases and Output

Test Case	Input Given	Output
Test Case 1 (Normal Input)	Choice: 1 ID: E101 Type: Casual From: 2026-12-10 To: 2026-12-12	Leave approved: 3 days Casual | Balance Casual: 9
Test Case 2 (Full Balance)	Choice: 1 ID: E101 Type: Casual From: 2026-11-02 To: 2026-11-13	Leave approved: 12 days Casual | Balance Casual: 0
Test Case 3 (Invalid Dates)	Choice: 1 ID: E101 Type: Sick From: 2026-12-15 To: 2026-12-10	Invalid dates. To-date cannot be before from-date.
Test Case 4 (Rejected & Balance)	(a) Choice 1 | ID: E101 | Casual | From: 2027-01-01 | To: 2027-01-14 (b) Choice 2 | ID: E101	(a) Leave rejected. Requested: 14 days | Available: 12 days (b) Casual: 12 | Sick: 10 | Earned: 15

5. Console Output Screenshot Text

```
1. Apply Leave
2. View Balance
3. Exit
Enter choice: 1
Enter Employee ID: E101
Enter Leave Type (Casual/Sick/Earned): Casual
Enter From Date (YYYY-MM-DD): 2026-12-10
Enter To Date (YYYY-MM-DD): 2026-12-12
Leave approved: 3 days Casual | Balance Casual: 9

1. Apply Leave
2. View Balance
3. Exit
Enter choice: 2
Enter Employee ID: E101
Casual: 9 | Sick: 10 | Earned: 15
```